

AI知识点

什么是 skill?

Skill 是一种预设的指令或工作流，旨在让 AI 完成特定的任务（如 **brainstorming**、代码生成等）。它是可复用、可调用的自动化能力。

怎么调用？

三种调用方式：

命令行式调用：

输入 `/+ skill 名称`（前提是该技能有效，且位于全局路径 `~/.trae/skills` 或项目路径 `工程目录/.trae/skills/` 中）。

自然语言指名调用：

直接在对话中指明技能名称，例如“调用 **brainstorming** 技能”。

文件拖拽：

将 **skill.md** 文件直接拖拽到聊天框中。

怎么实现一个 skill?

可以通过“元技能”来实现。告诉 AI 你的需求，让它生成一个技能，或者直接调用 **skill-creator** 技能来完成技能的创建。

怎么获得技能？

使用 **skillhub** 或 **findskill** 功能来查找和获取。

你用过哪些比较好的技能？

推荐的技能包括 `superpowers`、`skill-creator`、`find-skill` 等。

技能存储路径

全局路径 (`~/trae/skills`): 存放在这里的技能对所有项目都有效，适合通用的、跨项目的辅助功能（如通用的代码规范检查、通用的文档生成）。

项目路径 (工程目录`/trae/skills/`): 存放在这里的技能仅对当前项目有效，适合特定于该项目的业务逻辑、特定框架的代码生成等。

Skill 的本质

在 Trae 中，Skill 通常是一个 Markdown 文件（如 `skill.md`）。这个文件内部包含了给 AI 的系统提示词、参数定义以及执行逻辑。

Skill-Creator: 这是一种“自举”或“元编程”的概念。利用 AI 编写 AI 的指令，通过描述需求，让 AI 自动生成符合规范的 Markdown 技能文件。

Skill Hub

类似于代码的包管理器（如 `npm`, `pip`）或应用商店。它是一个中心化的仓库，用户可以从其中搜索、下载别人写好的优秀 Skill，分享给社区使用。

交互方式

Slash Command (/): 这是现代 AI 编辑器（如 VS Code Copilot, Cursor, Trae）的标准交互模式，用于快速触发特定工具。

自然语言触发: 降低了使用门槛，用户不需要记忆具体的命令，只需要表达意图。

LLM (Large Language Model)

大语言模型本质上是一个基于海量文本数据训练得到的概率生成模型，他并不是“像人一样真正理解语言”的系统，而是在给定上下文后，学习“下一个token最可能是什么”。在形式上，它学习的是： $P(x^{**t} \mid x_1, x_2, \dots, x^{**t-1})$

也就是：在前面一串token已知的前提下，预测当前token的概率分布

因为：

- LLM的输出不是“查答案”，而是“按概率生成”

- 代码生成、代码补全、Bug修复等任务，本质上都可以转化为“给定上下文后继续生成更合理的token序列”
- 这解释了为什么模型会出现：幻觉、API虚构、局部合理但全局错误、长上下文中前后不一致

1.什么是概率生成模型

概率生成模型的核心不是硬编码规则，而是根据训练中观察到的数据模式，为候选输出分配概率

例如输入

```
int add(int a,int b)
{
    return ...
}
```

模型在return后可能认为高概率的读写是

a+b,sum(a,b),a-b

其中a+b的概率更高，因为训练数据中这个模式更常见、更合理

所以，LLM并不是“理解加法定义后推导出来的”，而是通过统计规律与模式抽象，给出最可能续写。

2.什么是基于上下文预测下一个token

可以理解为：模型每一步都看当前已有内容，然后决定接下来最合适的一个token，会根据上下文预测下一个token，这说明模型不是孤立的生成字符，而是在上下文条件下逐token生成。

3.为什么LLM能迁移到代码任务

代码看似与自然语言不同，但二者有一个关键共性：都是序列结构，都存在强上下文依赖。

代码中同样存在：语法模式、结构模式、命名模式、API调用模式、模块依赖关系、注释与实现之间的语义对应

比如下面的自然语言和代码都属于“上下文到续写”的问题：

自然语言：

今天天气很好，，我想去

高概率续写的可能是：

散步/公园/运动

代码：

```
numbers = [1,2,3]
total = 0
for n in numbers:
```

高概率续写可能是：

```
total += n
```

模型之所以能做代码任务，不是因为“代码 = 语言”，而是因为Transformer能处理序列中的长短程依赖，并在大量代码数据上学习到代码分布特征

4.为什么代码能力不是“真正学会编程”

LLM能生成代码，不代表它具备人类工程师意义上的完整编程能力。

它通常缺少：稳定的目标分解能力、对真实运行环境的完全感知、对依赖版本的实时确认、对业务约束的严格验证、对多文件、多服务系统的全局一致性保证

所以他的准确定位是：“概率驱动的代码生成和辅助推理工具，而不是“完全自主的软件工程师”

Token

Agent Skill 渐进式披露

元数据层：存放名称（name）和描述（description），这些数据在agent中会始终加载

指令层：在SKILL.md中，存放除名称和描述之外的内存容，这些数据是按需加载

资源层：分为Reference和Script

Reference： agent中被读取，所以必定会占据上下文消耗token，这也是按需加载（比如只有在涉及到某些关键词时才会加载）

Script： 就是执行的代码段，这个一般不会读取，一般不会占用token，agent只关心执行的结果，不关心里面的内容（当然如果描述的需求不够清晰，agent也是可能会看一眼里面的内容的）

Skill 与 Mcp的区别

MCP： 它的功能是给大模型提供数据，比如查询昨天的销售记录、读取订单的物流状态，它本质上是一个独立的程序，重量级，安全，稳定

Skill： 它的功能是教大模型如何处理数据，比如会议总结必须要有议题、汇报文档必须包含数据，它本质上是一个文档，轻量级，不太安全

MCP和Skill适用于不同的场景，一些场景下联合起来用可能效果更好

大模型幻觉

定义：模型生成看似合理但实际不正确或与事实不符的内容

产生原因：

- 1.训练数据中的噪声和错误
- 2.模型倾向于生成流畅但不准确的文本
- 3.缺乏外部知识验证

4.长尾知识覆盖不足

解决方案：

1.RAG（检索增强生成）：引入外部知识库

2.RLHF：人类反馈强化学习

3.Chain-of-Thought：引导逐步推理

4.Self-Consistency：多次采样取一致答案

5.知识图谱增强

6.事实核查后处理

Prompt Engineering（提示工程）

Prompt Engineering 是一种通过【设计和优化输入提示（prompt）】来引导大语言模型（LLM）产生更准确、更符合预期输出的技术和方法论。

简单来说：你怎么问，决定了模型怎么答

六大核心技巧

1.Zero-shot / Few-shot（示例驱动）

- Zero-shot：不给示例，直接提问。适合简单任务
- Few-shot：提示2-5个输入-输出范例，让模型“照猫画虎”，比纯文字描述更有效

2.Chain-of-Thought（CoT）（思维链）

- 加上“Let's think step by step”，引导模型逐步推理
- 对数学、逻辑推理类任务效果显著
- 可以配合Few-shot使用（给几条带推理过程的范例）

3.Role Prompting（角色设定）

- 比如说，“你是一个资深的算法工程师，擅长用通俗语言解释复杂概念”
- 约束输出风格、专业深度和表达方式

4.ReAct（推理 + 行动循环）

- Thought（思考）-> Action（执行工具）->Observation（观察结果）->循环
- 是AI Agent的核心范式，让模型能使用搜索引擎、代码执行等工具

5.Self-Consistency（多次采样投票）

- 对同一问题多次采样，取出现最多次的答案
- 本质是用“多投少”提示可靠性

6.Tree-of-Thought（ToT）（思维树搜索）

- 搜索多条推理路径，允许回溯，选择最优路径
- 比线性推理（CoT）更适合需要“试探”的复杂问题

核心原则

- 1.明确性：指令越具体，输出越可控
- 2.结构化：用分隔符、模板组织输入输出
- 3.示例引导：Few-shot比纯描述更有效
- 4.分步推理：复杂问题拆解为子步骤
- 5.迭代优化：根据输出反复调整Prompt
- 6.约束输出：指定格式、长度、语言
- 7.角色代入：赋予专业身份提升回答质量
- 8.负面提示：明确说明不要做什么

Prompt 的定义

定义及解释

Prompt 不是简单的“提示词”，也不是随便输入给模型的一句话。在工程语境里，Prompt 更准确的定义应是：

任务表达接口：人把任务交给模型的接口层。

行为约束手段：通过文字、结构、格式要求约束模型输出。

需求到执行的桥梁：把模糊的人类需求转化为模型更容易执行的任务描述。

因此，**Prompt** 的核心不是“说了什么”，而是：

是否把任务说清楚；

是否给了必要上下文；

是否控制了输出边界；

是否让结果可验证、可复用、可集成。

很多初学者把 **Prompt** 理解为“问模型一个问题”，这在聊天场景里勉强成立，但在 **AI** 编程、**Agent** 场景中远远不够。因为工程任务需要的是稳定、约束、结构化、可执行的输入，而不是自然随意的表达。

核心原理

一个有效 **Prompt** 往往至少隐含以下结构：

目标：你希望模型做什么

背景：模型需要知道什么

约束：模型不能做什么、必须遵守什么

输出：最终交付格式是什么

验证标准：什么结果算完成

可以把 **Prompt** 理解为“面向大模型的任务规格说明”。

在 **AI** 编程 / 代码任务中的体现

在代码场景里，**Prompt** 的质量直接影响：

代码是否符合技术栈

是否遵守接口定义

是否考虑边界条件

是否便于测试和合并

是否能稳定输出指定格式

例如：

模糊写法

这个 **Prompt** 看似能用，但模型会自己猜：

用什么语言？

用什么框架？

返回格式是什么？

是否需要 JWT（JSON Web Token）？

是否要接数据库？

错误码如何定义？

工程化写法

后者才更接近“可执行任务描述”。

企业中的典型使用场景

企业内部代码助手生成接口代码

AI 辅助重构旧代码

智能测试生成

智能 SQL 辅助查询

Copilot 根据仓库规范输出 **patch**

Agent 按企业规则调用工具并返回结构化结果

在这些场景中，Prompt 已经不是“聊天内容”，而是系统能力设计的一部分。

常见误区 / 风险边界

误区 1: Prompt 就是多写一点字 不是。Prompt 强调的是结构，而不是长度。

误区 2: 模型强就不需要 Prompt 不对。模型能力决定上限，但 Prompt 决定任务落地质量。

误区 3: 自然语言表达越口语越好 在代码任务里，通常越结构化越好。

风险边界

Prompt 不能替代真实上下文

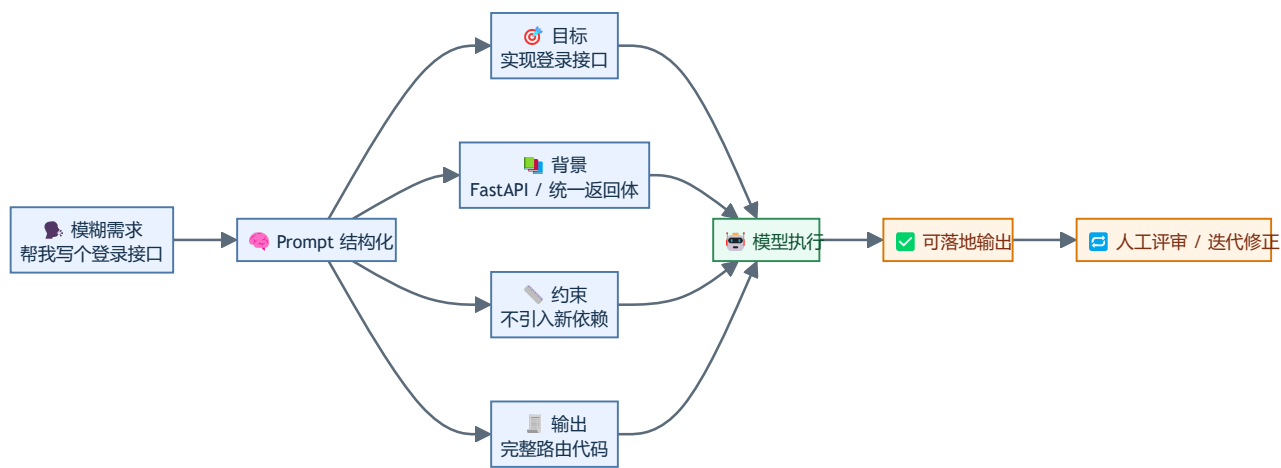
Prompt 不能弥补仓库信息缺失

Prompt 不能自动解决冲突指令和不可信输入问题

表格：自然语言需求 vs 可执行 Prompt

维度	自然语言需求	可执行 PROMPT
表达目的	传达想法	驱动模型执行
目标清晰度	常模糊	明确且可验证
背景信息	常省略	显式补充
约束条件	很少说明	明确边界
输出要求	随意	可解析、可验收
工程价值	沟通为主	落地为主

图 1：Prompt 的本质：从模糊需求到可执行任务描述



图解说明

左侧是人的原始需求，通常不够执行化。

中间是 **Prompt** 的结构层，把任务变成“目标 + 背景 + 约束 + 输出”。

右侧是模型执行与反馈闭环。

这也是 **Prompt** 工程区别于普通聊天的本质。

Prompt 在 AI 编程中的作用

定义及解释

Prompt 在 AI 编程中的作用，不是单纯“让模型写代码”，而是：

指定代码任务目标

约束技术选型与边界

提供必要上下文

稳定输出格式

降低返工成本

同一个模型在不同 **Prompt** 下，输出质量往往差异巨大。原因不是模型突然变聪明或变笨，而是输入任务规格不同。

核心原理

模型在代码任务中本质上是在做条件生成。**Prompt** 质量越高，模型的“搜索空间”越小，越容易命中符合项目要求的解。

Prompt 会显著影响以下方面：

正确性：是否实现了真实需求

完整性：是否遗漏异常处理、边界条件

可维护性：命名、结构、模块拆分是否合理

一致性：是否符合仓库风格

安全性：是否引入危险做法

稳定性：多次运行结果是否接近

在 AI 编程 / 代码任务中的体现

代码生成

Prompt 决定使用什么框架、接口形式、返回格式。

代码重构

Prompt 决定是否保留原行为、是否输出 **diff**。

调试

Prompt 决定模型是泛泛猜测，还是基于日志与代码定位问题。

测试生成

Prompt 决定覆盖 **happy path**，还是覆盖边界与异常。

为什么同一个模型会因为 **Prompt** 质量不同而输出差异很大

因为模型不是“自动理解你的意图”，它只能依据你提供的任务描述和上下文进行条件预测。缺信息时，它会猜；猜得多，偏差就大。

例如同样是“写一个缓存模块”：

Prompt A

可能输出：

Python / JavaScript 任意一种

内存缓存 / **Redis** 任意一种

无过期策略

无线程安全考虑

无接口约定

Prompt B

明显更可控。

企业中的典型使用场景

代码补全助手：根据当前文件和函数签名补全实现

代码审查助手：按企业规范输出审查项

测试生成助手：按项目测试框架生成单测

CI 修复助手：根据日志与 `patch` 建议修复

数据分析助手：受限生成只读 SQL

示例：测试生成 低质量 Prompt：

给这个函数写测试

高质量 Prompt：

请为下面的 Python 函数生成 `pytest` 单元测试，要求：

- \1. 覆盖正常输入、边界输入、异常输入；
- \2. 若函数依赖外部服务，请使用 `mock`；
- \3. 输出一个完整 `test_xxx.py` 文件；
- \4. 不修改原函数实现；
- \5. 最后列出测试覆盖点清单。

常见误区 / 风险边界

误以为“模型写出来能跑”就算成功 实际上企业更关注是否符合架构、规范、接口和可维护性。

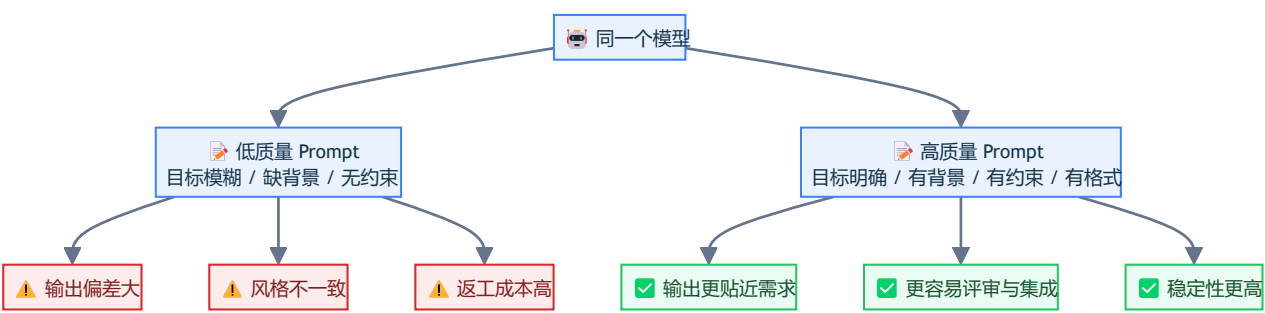
误以为 **Prompt** 只影响语言风格 实际上它直接影响代码逻辑与边界。

误以为代码任务只需描述功能 真实工程任务还要给约束、上下文、输出格式。

表格：代码任务类型 vs **Prompt** 影响维度

代码任务	PROMPT 重点	常见失败	修复方向
代码生成	技术栈、接口、输出	乱选框架、遗漏边界	明确环境与返回格式
代码重构	行为不变、修改范围	改坏兼容性	增加“行为不变”约束
调试	日志、复现、相关代码	空泛猜测根因	补充报错和上下文
测试生成	覆盖目标、框架、mock	只测 happy path	指定边界与异常分支
SQL / Shell	安全边界、环境	误删、误写、高风险操作	显式只读与确认机制

图 2：Prompt 质量如何影响 AI 编程结果



图解说明

模型不变，差别来自 **Prompt**。

高质量 **Prompt** 通过降低模型的自由猜测空间，提高工程可用性。

这也是企业投入 **Prompt** 工程的直接原因。